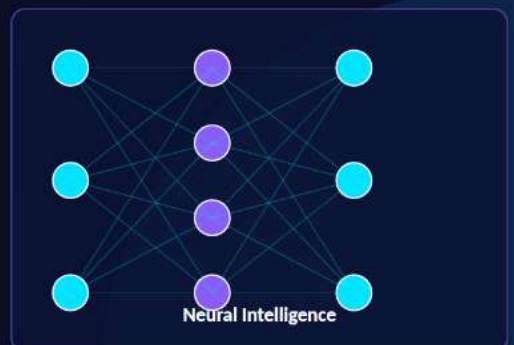


AI-Powered Credit Card Fraud Detection

• using CTGAN-Based Synthetic Data Generation

PRESENTED BY	Sneha Verma
COMPANY	GNCIPL
INTERNSHIP DOMAIN	Artificial Intelligence & Machine Learning
PROJECT DOMAIN	Generative AI for Enterprise Applications
PLATFORM	Google Colab
LANGUAGE	Python
DURATION	6-Week AI/ML Internship



Agenda

01

Introduction

02

Problem Statement

03

Objectives

04

Dataset

05

Generative AI

06

CTGAN Architecture

07

Methodology

08

Exploratory Data
Analysis

09

Data Preprocessing

10

Synthetic Data
Generation

11

Data Augmentation

12

ML Models

13

Model Evaluation

14

Results & Comparison

15

Future Scope

16

Conclusion

Introduction



What is Credit Card Fraud?

Unauthorized use of card details to make illegitimate transactions.



Why Detection Matters

Banks lose billions annually; customer trust is critical.



Bank Challenges

Volume, velocity, and evolving fraud patterns overwhelm rule engines.



Role of AI

Learns patterns beyond static rules and adapts to new threats.



Role of Machine Learning

Classifies transactions using historical labelled data.



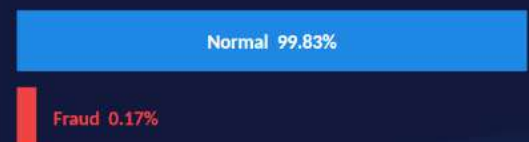
Need for Generative AI

Creates realistic synthetic fraud to balance training data.

Problem Statement

- ! Very few fraud transactions in original dataset
- ! Highly imbalanced class distribution
- ! Traditional ML models become biased towards majority class
- ! Poor recall on the fraud (minority) class
- ! Need to synthetically generate more fraud samples

CLASS IMBALANCE



OUTPUT SCREENSHOT

```
Class
0.0    92967
1.0      213
Name: count, dtype: int64

Normal Transactions : 92967
Fraud Transactions  : 213

df["Class"].value_counts()
```

Project Objectives



Generate Synthetic Fraud

Use CTGAN to create realistic fraud samples



Reduce Class Imbalance

Rebalance dataset for fair learning



Improve Fraud Detection

Boost recall on the minority class



Train ML Models

Random Forest & XGBoost pipelines



Compare Performance

Benchmark both models on key metrics



Evaluate End-to-End

Accuracy, Precision, Recall, F1, ROC-AUC

Why Generative AI?

Generative AI

Models that learn data distributions and generate new realistic samples — text, images, or tabular data.

- Random Oversampling → duplicates data, causes overfitting
- Undersampling → discards valuable majority information
- SMOTE → linear interpolation, weak for complex tabular data
- CTGAN → learns conditional distributions of tabular data

TECHNIQUE COMPARISON

Technique	Realism	Balance	Verdict
Random Oversampling	Low	Yes	Overfit
Undersampling	N/A	Yes	Info Loss
SMOTE	Medium	Yes	Limited
CTGAN	High	Yes	Best

★ CTGAN chosen for its ability to model complex tabular distributions.

CTGAN — Conditional Tabular GAN

Core Components

Generator

Produces synthetic samples from noise + condition vector.

Discriminator

Distinguishes real from generated tabular rows.

Conditional Learning

Samples respect the minority class distribution.

Training Process

Adversarial optimization until equilibrium.

Generation

Sample 5,000 realistic synthetic fraud rows.



Dataset Overview

Credit Card Fraud Detection

Source: Kaggle

284,807

Total Rows

492

Fraud Records

31

Columns

0.17%

Fraud Ratio

Features

- Time
- Amount
- V1 – V28 (PCA)
- Class (0 = Normal, 1 = Fraud)

OUTPUT SCREENSHOT

```
Dataset Loaded Successfully
Time      V1      V2      V3      V4      V5      V6      V7
0  0 -1.359807 -0.072781 2.536347 1.378155 -0.338321 0.462388 0.239599
1  0  1.191857 0.266151 0.166480 0.448154 0.060018 -0.082361 -0.078803
2  1 -1.358354 -1.340163 1.773209 0.379780 -0.503198 1.800499 0.791461
3  1 -0.966272 -0.185226 1.792993 -0.863291 -0.010309 1.247203 0.237609
4  2 -1.158233 0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941

      V8      V9      ...      V21      V22      V23      V24      V25
0  0.098698 0.363787 ... -0.018307 0.277838 -0.110474 0.066928 0.128539
1  0.085102 -0.255425 ... -0.225775 -0.638672 0.101288 -0.339846 0.167170
2  0.247676 -1.514654 ... 0.247998 0.771679 0.909412 -0.689281 -0.327642
3  0.377436 -1.387024 ... -0.108300 0.005274 -0.190321 -1.175575 0.647376
4 -0.270533 0.817739 ... -0.009431 0.798278 -0.137458 0.141267 -0.206010

      V26      V27      V28      Amount      Class
0 -0.189115 0.133558 -0.021053 149.62 0.0
1  0.125895 -0.008983 0.014724  2.69 0.0
2 -0.139097 -0.055353 -0.059752 378.66 0.0
3 -0.221929 0.062723 0.061458 123.50 0.0
4  0.502292 0.219422 0.215153  69.99 0.0

[5 rows x 31 columns]
```


Exploratory Data Analysis

EDA Steps

- 1 Checking Missing Values
- 2 Checking Duplicate Values
- 3 Class Distribution
- 4 Statistical Summary
- 5 Understanding Feature Ranges

OUTPUT SCREENSHOT

```
V21      1
V22      1
V23      1
V24      1
V25      1
V26      1
V27      1
V28      1
Amount   1
Class    1
dtype: int64
```

OUTPUT SCREENSHOT

```
Shape of Dataset: (93181, 31)
Columns:
Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18',
       'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28',
       'Class'],
      dtype='object')
```

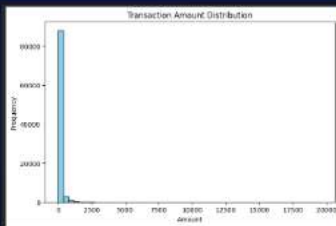
```
df.duplicated().sum()
```

OUTPUT SCREENSHOT

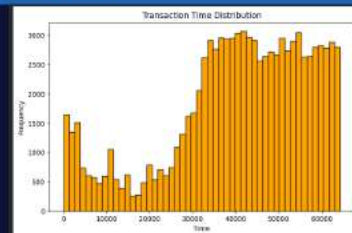
```
***
      Time      V1      V2      V3      V4      V5      V6      V7      V8      V9  ...
count  93181.000000  93181.000000  93181.000000  93181.000000  93181.000000  93181.000000  93181.000000  93181.000000  93181.000000  93181.000000  ...
mean   40720.460495  -0.263002  -0.040780   0.676305   0.163064  -0.280076   0.993917  -0.100816   0.055540  -0.036696  ...
std    16392.444505   1.869011   1.664309   1.341575   1.355105   1.360667   1.303615   1.226238   1.209672   1.123316  ...
min      0.000000  -56.407510  -72.715728  -33.630964  -5.172595  -42.147898  -26.160006  -31.754946  -73.216718  -9.283925  ...
25%   32678.000000  -1.028015  -0.605705   0.179018  -0.716783  -0.898706  -0.647592  -0.600859  -0.139122  -0.696859  ...
50%   42848.000000  -0.258489   0.072818   0.756532   0.189117  -0.314508  -0.156636  -0.069893   0.072409  -0.105294  ...
75%   53622.000000   1.153021   0.728956   1.381324   1.034477   0.250915   0.487658   0.415870   0.357253   0.576494  ...
max    64283.000000   1.960497   18.902453   4.226108  16.715937   34.801686  22.520298  36.677268  20.007208  10.392889  ...
8 rows x 31 columns
```

EDA Visualizations

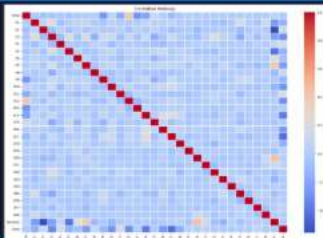
OUTPUT SCREENSHOT



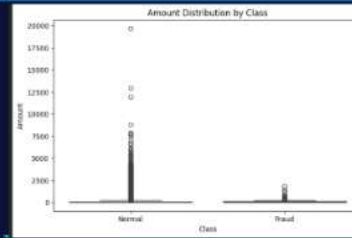
OUTPUT SCREENSHOT



OUTPUT SCREENSHOT



OUTPUT SCREENSHOT



Data Preprocessing

Preprocessing Workflow

1 Duplicate Removal

2 Scale Amount

3 Scale Time

4 Extract Fraud Data

5 Prepare for CTGAN

OUTPUT SCREENSHOT

Dataset Loaded Successfully

```
Time      V1      V2      V3      V4      V5      V6      V7
0  0 -1.359887 -0.072781 2.536347 1.178155 -0.338321 0.462388 0.239599
1  0 1.191857 0.266151 0.166488 0.448154 0.060018 -0.082161 -0.078801
2  1 -1.358354 -1.340163 1.773209 0.379780 0.503198 1.008499 0.791461
3  1 0.968272 -0.185228 1.792993 0.363291 -0.010309 1.247203 0.257609
4  2 -1.158213 0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941

V8      V9      ...      V21      V22      V23      V24      V25
0 0.098698 0.363787 ... -0.018307 0.277838 -0.110474 0.066028 0.128539
1 0.085102 -0.255425 ... -0.225775 -0.638672 0.101288 -0.339846 0.167170
2 0.247676 -1.514654 ... 0.247998 0.771679 0.909412 -0.689281 -0.327642
3 0.377436 -1.387024 ... -0.108300 0.005274 -0.190321 -1.175575 0.647376
4 -0.270533 0.817739 ... -0.009431 0.798278 -0.137458 0.141267 -0.206010

V26      V27      V28      Amount      Class
0 -0.189115 0.133558 -0.021053 149.62 0.0
1 0.125895 -0.000983 0.014724 2.69 0.0
2 -0.139097 -0.055353 -0.059752 378.66 0.0
3 -0.221929 0.062723 0.061458 123.50 0.0
4 0.502292 0.219422 0.215153 69.99 0.0
```

[5 rows x 31 columns]

Preprocessed Dataset Head

`df.head()`

Synthetic Data Generation

Generation Pipeline

- Fraud records extracted
- CTGAN trained on fraud only
- Learns fraud distribution
- 5,000 synthetic samples generated
- Synthetic data saved to disk

OUTPUT SCREENSHOT

Fraud Data Shape: (213, 31)

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V22	V23	V24
541	-2.312227	1.951992	-1.609851	3.997906	-0.522188	-1.426545	-2.537387	1.391657	-2.770089	-2.772272	...	-0.035049	-0.465211	0.320198
623	-3.043541	-3.157307	1.088463	2.288644	1.359805	-1.064823	0.325574	-0.067794	-0.270953	-0.838587	...	0.435477	1.375966	-0.293803
4920	-2.303350	1.759247	-0.359745	2.330243	-0.821628	-0.075788	0.562320	-0.399147	-0.238253	-1.525412	...	-0.932391	0.172726	-0.087330
6108	-4.397974	1.358367	-2.592844	2.679787	-1.128131	-1.706536	-3.496197	-0.248778	-0.247768	-4.801637	...	0.176968	-0.436207	-0.053502
6329	1.234235	3.019740	-4.304597	4.732795	3.624201	-1.357746	1.713445	-0.496358	-1.282858	-2.447469	...	-0.704181	-0.656805	-1.632653

5 rows x 31 columns

`fraud_data.head()`

OUTPUT SCREENSHOT

`fraud_data.shape`

OUTPUT SCREENSHOT

Synthetic Data Shape: (5000, 31)

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V22
0	-8.741009	2.594943	-10.324794	-0.831427	-0.754834	-1.096500	-1.241739	-3.2177535	-2.828721	-12.737242	...	-0.607749
1	1.296231	3.163077	-20.400909	0.805631	2.063301	-1.473339	-4.764988	-2.236230	1.249243	0.907552	...	-0.880886
2	1.296231	4.552817	-0.852324	5.254442	1.146922	-1.383280	-4.594393	-0.533734	-1.317511	-4.916185	...	-1.035057
3	0.706296	4.297314	2.041003	2.585462	5.269255	0.674304	0.763873	-0.417853	-3.265152	-0.494841	...	-0.195537
4	0.873296	0.023637	-2.244294	0.914074	5.289255	1.732086	-3.674137	4.006155	-5.984022	-0.000851	...	-1.147165

5 rows x 31 columns

`synthetic_data.head()`

OUTPUT SCREENSHOT

`synthetic_data.shape`

Data Augmentation

Original Dataset

Synthetic Fraud

Augmented Dataset

Benefits

- Reduced Class Imbalance
- Better Model Learning
- More Fraud Examples

OUTPUT SCREENSHOT

```
Augmented Dataset Shape: (97853, 31)

# Remove any missing values
augmented_data = augmented_data.dropna().reset_index(drop=True)

print("Missing Values:")
print(augmented_data.isnull().sum().sum())

print("Duplicate Rows:", augmented_data.duplicated().sum())

Missing Values:
0
Duplicate Rows: 0
```

OUTPUT SCREENSHOT

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)

Training Shape: (78282, 30)
Testing Shape: (19571, 30)
```

Machine Learning Models

Environment

PLATFORM

Google Colab

LANGUAGE

Python

Libraries

Pandas

NumPy

Matplotlib

Scikit-learn

XGBoost

SDV CTGAN

Joblib

Training Pipeline

Augmented Data



80 / 20 Train-Test Split



Random Forest | XGBoost



Prediction



Evaluation

Model Evaluation

Metrics

Accuracy

Precision

Recall

F1 Score

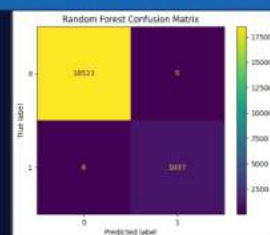
ROC-AUC

Confusion Matrix

OUTPUT SCREENSHOT

Accuracy : 0.9994
Precision : 0.9952
Recall : 0.9942
F1 Score : 0.9947
ROC AUC : 0.998

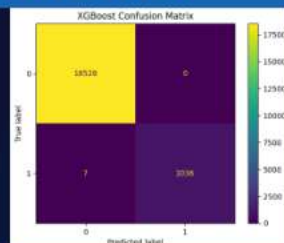
OUTPUT SCREENSHOT



OUTPUT SCREENSHOT

Accuracy : 0.9996
Precision : 1.0
Recall : 0.9933
F1 Score : 0.9966
ROC AUC : 0.9999

OUTPUT SCREENSHOT



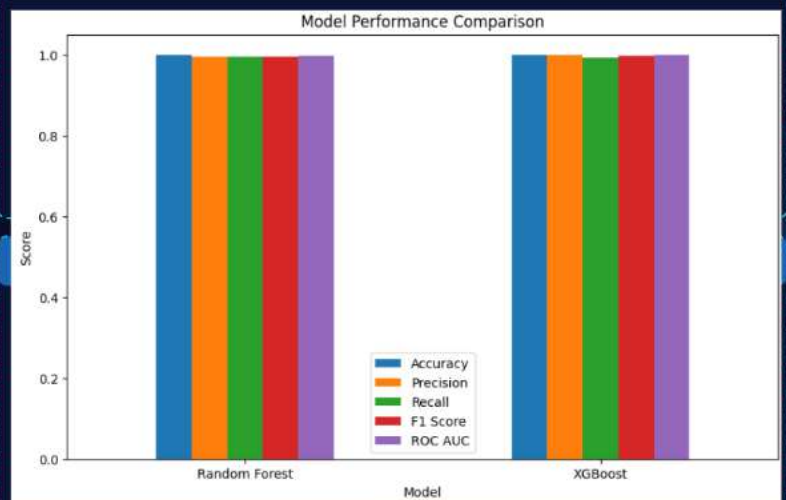
Results Comparison

MODEL METRICS

Metric	Random Forest	XGBoost
Accuracy	—	—
Precision	—	—
Recall	—	—
F1 Score	—	—
ROC-AUC	—	—

Values to be filled from Google Colab results.

OUTPUT SCREENSHOT



Complete Project Workflow



Key Achievements



Successfully Implemented CTGAN



Generated 5,000 synthetic fraud transactions



Reduced class imbalance



Improved fraud detection performance



Compared Random Forest vs XGBoost



Implemented complete AI/ML pipeline



Developed end-to-end in Google Colab



Prepared professional GitHub project

Future Scope

01

Real-Time Fraud Detection

02

Web Application

03

Flask / FastAPI Deployment

04

Cloud Deployment

05

Deep Learning Models

06

Explainable AI (XAI)

07

Banking Integration

08

Continuous Learning

09

Enterprise Deployment

Roadmap towards a production-grade, enterprise fraud detection platform.

THANK YOU

AI-Powered Credit Card Fraud Detection using CTGAN

PRESENTED BY

Sneha Verma

COMPANY

GNCIPL

INTERNSHIP DOMAIN

Artificial Intelligence & Machine Learning

PLATFORM

Google Colab

LANGUAGE

Python

"Thank you for your valuable time and attention. I welcome your questions and feedback."

Building Smarter Fraud Detection Systems with Generative AI